BLOODCONNECT 2.0: AN ADAPTIVE EMERGENCY BLOOD-DONOR COORDINATION SYSTEM

23OED971 – INTRODUCTION TO DESIGN THINKING REPORT

Submitted by

FREDDY MATTHEW R	**9124104041**
BOOPATHI P R	**9124104026**
BALAKUMARAN E	**9124104020**
ABILASH ALEXANDER E	**9124104004**
HARISANKARAN J	**9124104048**
JOSHUA BENNET	**9124104063**

in the partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

KCG COLLEGE OF TECHNOLOGY, KARAPAKKAM

(AUTONOMOUS)

NOVEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**BloodConnect 2.0: An Adaptive Emergency Blood-Donor Coordination System**" is being submitted by **Freddy Matthew R (9124104041)**, **Boopathi P R (9124104026)**, **Balakumaran E (9124104020)**, **Abilash Alexander E (9124104004)**, **Harisankaran J (9124104048)** and **Joshua Bennet (9124104063)**. The work presented in this report was carried out as part of **23OED971 – Introduction to Design Thinking** under our guidance and supervision during the academic year 2026–2027.

SIGNATURE OF THE HOD

Dr. S. Revathy
Professor & Head
Department of CSE
KCG College of Technology
Chennai - 600 097

SIGNATURE OF THE SUPERVISOR

Mr. Ravi Karthik R
Assistant Professor
Department of CSE
KCG College of Technology
Chennai - 600 097

Submitted for the Introduction to Design Thinking presentation held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to sincerely thank all those who have supported and guided us throughout the successful completion of this project. First and foremost, we express our heartfelt gratitude to **Dr. G. Prabhakaran**, Principal, KCG College of Technology, for providing the necessary facilities and encouragement to carry out this project.

We extend our sincere thanks to the Head of the Department, **Dr. S. Revathy**, for her continuous support and motivation.

We are deeply grateful to our Course Coordinator, **Mr. Ravi Karthik R**, for his valuable suggestions, constant encouragement, feedback and timely guidance throughout the project work.

We sincerely thank the doctors who kindly spared their valuable time for our interviews and shared their experience of arranging blood for patients in emergencies. Their insights shaped the direction of this project. We also thank all the faculty members and staff of the Department of Computer Science and Engineering for their academic and administrative support.

Finally, we would like to thank our family and friends for their unconditional support, encouragement, and belief in us throughout this journey.

Freddy Matthew R	**9124104041**
Boopathi P R	**9124104026**
Balakumaran E	**9124104020**
Abilash Alexander E	**9124104004**
Harisankaran J	**9124104048**
Joshua Bennet	**9124104063**

ABSTRACT

In a medical emergency, blood is often arranged by contacting voluntary donors. In practice the request is forwarded to as many people as possible and then nobody keeps track of who actually answered. Coordinators cannot see whether enough volunteers have agreed, donors receive alerts for requests that are already covered, and there is no record of why a particular donor was contacted. BloodConnect 2.0 is a web application that turns this one-shot “alert everyone” approach into a managed, response-aware coordination process.

The project follows the Design Thinking process. In the Empathize stage the team interviewed three doctors using a set of 20 questions. They described delays when the required blood group is unavailable, difficulty with rare groups such as AB-negative, and dependence on social media and personal contacts. The insights were used to define How Might We questions, brainstorm forty ideas and select three concepts, build a working prototype and test it. The system extends an existing Django and React platform with an Adaptive Emergency Escalation Engine. For each approved request it ranks eligible donors with a transparent score, alerts a small first batch, records accepted, declined and unanswered alerts, recalculates the remaining volunteer need, and widens the search area in stages only when earlier responses are not enough. Every decision is stored with its reason on a timeline.

A simulator with five scenarios and fictional donors allows safe testing, and a comparison module measured the engine against two baselines (basic broadcast and fixed-interval escalation) over five seeded runs per scenario. In the simulated high-response scenario the adaptive strategy contacted about 10 donors against about 42 for broadcast, but took about 11 minutes against about 4 to reach the volunteer target. The system passes all 116 automated tests. All results are simulation-based and do not represent real hospital performance or donation outcomes.

Keywords: Design Thinking, Emergency Blood Donation, Donor Coordination, Adaptive Escalation, Django, React

TABLE OF CONTENTS

SECTION	PAGE
Bonafide Certificate	**ii**
Acknowledgement	**iii**
Abstract	**iv**
Table of Contents	**v**
List of Figures	**vii**
List of Tables	**viii**
CHAPTER 1 – INTRODUCTION	**1**
1.1 General	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Existing System	3
1.6 Proposed System	3
CHAPTER 2 – EMPATHY MAPPING	**4**
2.1 Empathize	4
2.1.1 Interviews Conducted	4
2.1.2 Key Insights	6
2.1.3 Empathy Map	7
2.2 Define	7
2.3 Ideate	8
2.4 Prototype	11
2.5 Test	11
2.6 System Overview	12
2.7 User Roles	13
2.8 System Workflow	13
2.9 Escalation Algorithm	14
2.9.1 Target and Remaining Need	14
2.9.2 Eligibility and Candidate Scoring	14
2.9.3 Search Stages and Response Windows	15
2.9.4 Batch Size	15
2.9.5 Decision Cycle	16
2.10 Main Modules	17
2.11 Database Design	18

CHAPTER 3 – RESULTS AND DISCUSSION **20**
3.1 Implementation Overview 20
3.2 User Interface 20
3.3 Working Procedure 30
3.4 Evaluation and Testing 31
3.5 Simulation Results 32
3.6 Discussion 33
3.7 Limitations 34
3.8 Expected Outcomes 34
CHAPTER 4 – CONCLUSIONS **35**
4.1 Conclusion 35
4.2 Future Enhancements 35
4.3 Team Contributions 36
4.4 Learning Outcomes 36
REFERENCES **37**

LIST OF FIGURES

Figure 2.1	The Five Stages of the Design Thinking Process	4
Figure 2.2	Interviews with Dr. Chanthru, Dr. Bruno and Dr. Selvam	6
Figure 2.3	Empathy Map (Stanford d.school Format)	7
Figure 2.4	Brainstorming Chart: Divergent and Convergent Thinking	9
Figure 2.5	Mind Map and the Three Selected Concepts	10
Figure 2.6	System Architecture of BloodConnect 2.0	13
Figure 2.7	System Workflow	14
Figure 2.8	Decision Cycle of the Escalation Engine	17
Figure 2.9	Entity-Relationship Diagram	19
Figure 3.1	Landing Page and Sign-in Page	21
Figure 3.2	Submit Emergency Blood Request (Requester)	22
Figure 3.3	Volunteer Donor Portal	23
Figure 3.4	Administrator Dashboard	24
Figure 3.5	Emergency Command Center	25
Figure 3.6	Outreach & Escalation: Metrics, Pipeline and Required Action	26
Figure 3.7	Candidate Score Breakdown and Decision Support	27
Figure 3.8	Live Event Timeline and Emergency Map	28
Figure 3.9	Emergency Response Simulator	28
Figure 3.10	Strategy Comparison (High Response Scenario)	29
Figure 3.11	Escalation Policy Settings	30
Figure 3.12	Donors Contacted and Time to Target by Strategy	33

LIST OF TABLES

Table 2.1	User Interview Questions	5
Table 2.2	Interview Answers and Key Insights	6
Table 2.3	User Roles	13
Table 2.4	Candidate Scoring Criteria (default weights)	15
Table 2.5	Default Search Stages	15
Table 2.6	Main Modules of BloodConnect 2.0	17
Table 2.7	Main Database Tables	18
Table 3.1	Technology Stack	20
Table 3.2	Automated Test Suite	31
Table 3.3	Functional Test Cases and Results	31
Table 3.4	Strategy Comparison (mean of 5 seeded runs, fictional donors)	32
Table 4.1	Team Members and Roles	36

CHAPTER 1
INTRODUCTION

1.1 General

Blood cannot be manufactured. Hospitals depend on blood banks and, when stock of a particular group or component is short, on voluntary donors who are willing to come in at short notice. Accidents, surgery, childbirth complications and conditions that need repeated transfusion all create requests that are urgent and must be met before a fixed time.

In Chennai, as in most Indian cities, the usual way to find donors in an emergency is informal. A family member or a hospital desk forwards a message through messaging groups and personal contacts, and people call back if they can help. The request travels widely, but nobody tracks whether enough people actually agreed, many donors receive alerts for requests that were solved hours earlier, and the coordinator has no clear picture of what to do next.

BloodConnect is a web platform built by our team for emergency blood-donor coordination across Chennai localities. It has three kinds of users (requesters, donors and administrators). The first version could find compatible donors and alert them, but it alerted everyone at once and then stopped. BloodConnect 2.0, the subject of this report, adds an Adaptive Emergency Escalation Engine that keeps managing each emergency after the first alert.

1.2 Problem Statement

Interviews with doctors showed that hospitals first check the blood bank and then contact donors through social media and personal contacts, and that delays happen when the required blood group, especially a rare one, is not available. There is no centralised donor system, and once a request is sent out nobody follows up on who has actually agreed. The problem addressed in this project is stated as the following How Might We question:

How might we make sure enough nearby eligible donors respond to an emergency blood request in time, without alerting every donor at once and while keeping every decision explainable?

1.3 Objectives

The main objectives of BloodConnect 2.0 are:

- To track every donor's response to each alert as accepted, declined or no response.
- To recalculate the remaining volunteer need after every response.
- To alert the nearest, best-ranked donors first and widen the search area only when needed.
- To explain, in plain language, why each donor was contacted and why each decision was taken.
- To give administrators a single Command Center for all active emergencies.
- To provide a safe simulator and a quantitative comparison against simpler strategies.
- To protect privacy and avoid any clinical claim about compatibility or eligibility.

1.4 Scope of the Project

The scope of BloodConnect 2.0 covers emergency donor coordination within Chennai. The project focuses on:

- Requester, donor and administrator accounts with role-based access.
- Blood requests with group, component, units, hospital, urgency and required-by time.
- Administrator verification before any donor is alerted.
- Staged, response-aware outreach with explainable candidate ranking.
- An event timeline, audit log and rule-based explanation assistant.
- A simulator and strategy comparison that use only fictional donors.
- Blood-bank stock and low-stock monitoring (carried over from version 1).

The coordination target is a number of *volunteers*, not blood units. "Alert sent" means an in-app notification record; SMS, e-mail, push, GPS and hospital-system

integrations are outside the current scope. The system does not decide clinical compatibility or donor eligibility; that remains with the blood bank or hospital.

1.5 Existing System

Outside the platform, emergency donor search is done through messaging groups, phone calls and social-media posts. Inside the platform, BloodConnect 1.x worked as follows: a requester created a request, an administrator approved it, and the system created an alert for every compatible donor at once. Donors could then accept or decline. The existing approach has the following limitations:

- Alerts are sent to everyone at once, causing alert fatigue for donors.
- Nobody tracks whether the number of people who said yes is enough.
- Unanswered alerts are not distinguished from refusals.
- There is no staged widening of the search area; it is all or nothing.
- There is no record of why a donor was contacted or why a decision was taken.
- There is no safe way to test or compare different outreach strategies.

1.6 Proposed System

BloodConnect 2.0 keeps the existing roles and request flow and adds an escalation engine behind it. When an administrator approves a request, the engine opens an emergency case, computes how many volunteers are needed (two per unit by default), ranks eligible donors and alerts only a small first batch. After every response it recalculates the remaining need and decides whether to wait, top up in the same area, expand to a wider area or ask an administrator to step in. The main advantages are:

- Fewer alerts per emergency when donors respond well.
- A live view of the remaining need for every emergency.
- Clear separation of accepted, declined and no-response (status unknown).
- Transparent, editable ranking weights and search stages.
- A full decision timeline and audit trail.
- A simulator and strategy comparison for safe, evidence-based evaluation.

CHAPTER 2

EMPATHY MAPPING

Design Thinking is a human-centred approach to solving problems. It begins with understanding the people who face the problem and continues through defining the problem, generating ideas, building a prototype and testing it. The five stages followed in this project are Empathize, Define, Ideate, Prototype and Test (Figure 2.1).

The Prototype and Test stages are described in Sections 2.4 and 2.5, the system design that supports the prototype is given in Sections 2.6 to 2.11, and the results obtained from the prototype are discussed in Chapter 3.

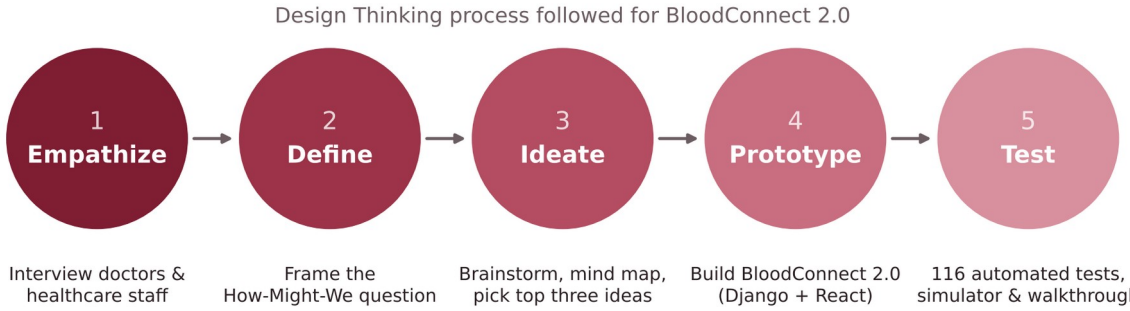


Figure 2.1: The Five Stages of the Design Thinking Process

2.1 Empathize

The Empathize stage focuses on understanding the experiences, needs and difficulties of the people who deal with emergency blood requests every day. Since doctors and hospital staff are the ones who arrange blood when a patient needs it urgently, the team chose to interview doctors working in hospitals and clinical laboratories in Chennai.

2.1.1 Interviews Conducted

A set of 20 open-ended questions was prepared (Table 2.1), using the “Five Whys” technique to dig deeper into the answers. The questions covered how often patients need blood urgently, the biggest challenges in arranging it, what the hospital does in an emergency, which blood groups are hardest to keep in stock, how donated

blood is made safe, donor eligibility, common misconceptions about donation, and what a blood donor application should offer.

Face-to-face interviews were conducted by the team members with three doctors: **Dr. Chanthru** (interviewed by Abilash Alexander), **Dr. Bruno** (interviewed by Balakumaran E) and **Dr. Selvam** (interviewed by Boopathi P R, Freddy Matthew R and Joshua Bennet). The interviews were video-recorded and the recordings are stored in a shared folder maintained by the team. Figure 2.2 shows frames from the recordings.

Table 2.1: User Interview Questions

#	Interview Question	#	Interview Question
1	How often do you encounter patients who urgently require blood transfusions?	11	Can a mobile app help find blood donors faster?
2	What is the biggest challenge in arranging blood for these patients?	12	What message would you like to share about blood donation?
3	Can you recall an incident where finding blood was particularly difficult? What happened?	13	Where should blood donation camps be held to get more participants?
4	What does the hospital do when a patient urgently needs blood?	14	What are the eligibility criteria for blood donation?
5	Why does it sometimes take time to arrange blood?	15	What are some common misconceptions that prevent people from donating blood?
6	Which blood group is the hardest to keep in stock?	16	What simple steps can colleges and communities take to encourage blood donation?
7	What challenges do hospitals face in maintaining an adequate blood supply?	17	How does blood play a vital role in the field of dentistry?
8	How do hospitals ensure that donated blood is safe for patients?	18	What advice would you give to someone who wants to become a regular blood donor?
9	What basic health checks are performed before a person donates blood?	19	What is the biggest improvement needed in India’s blood donation system?
10	Why is it important for hospitals to always have enough blood available?	20	If there were a blood donor mobile application, what feature would be most useful?

Each interview began with general questions about the doctor’s experience of emergency transfusions and then moved to specific incidents, the hospital’s procedure when blood is not available, and the checks performed before and after donation. Follow-up “why” questions were asked whenever an answer pointed to a delay or a difficulty.



Figure 2.2: Interviews with Dr. Chanthru, Dr. Bruno and Dr. Selvam

2.1.2 Key Insights

The answers were summarised after each interview and the recurring patterns were identified. The main insights are listed in Table 2.2.

Table 2.2: Interview Answers and Key Insights

S.No.	Summary of User Responses	Key Insight / Pain Point
1	The laboratory handles blood samples and blood units daily. The main challenge in providing compatible blood is maintaining availability and finding the required blood group when needed. Shortages or rare groups create difficulties during emergencies, which the laboratory team manages through coordination and the resources available.	Blood availability and finding the required group quickly is challenging, especially in emergencies and for rare groups.
2	Hospitals check blood banks or contact donors during emergencies, but delays occur when the required blood group is unavailable. A faster donor connection system can improve response time.	No centralised donor system; delays in emergencies; dependence on social media and personal contacts.
3	Mobile apps can help find blood donors quickly by connecting patients with nearby donors. Blood donation is a social responsibility, and camps should be held in colleges, schools, workplaces and public places. Donors must be healthy and meet age, weight and medical eligibility criteria.	Finding suitable donors quickly is challenging; technology and accessible camps can improve donor availability.
4	Timely blood availability is very important, especially during surgeries and emergencies. Fear, misconceptions and lack of awareness are common reasons why people hesitate to donate. Colleges and communities can create awareness and encourage young people to donate regularly.	Getting blood on time, and the lack of awareness and confidence among potential donors, remain major challenges.

S.No.	Summary of User Responses	Key Insight / Pain Point
5	When a patient urgently needs blood, the hospital checks the blood bank or contacts donors. Delays happen when the required group is unavailable. Rare groups such as AB-negative are harder to keep in stock.	Arranging compatible blood quickly is hard, especially for rare groups that are difficult to stock.
6	Donor fitness is checked before donation, and collected blood is screened to ensure it is safe. Hospitals must also manage limited supplies and sudden increases in demand.	Ensuring blood safety while keeping enough stock for emergencies is a major challenge.

2.1.3 Empathy Map

The findings were organised into an empathy map in the Stanford d.school format (Figure 2.3). It records the practical needs and pain points of the users, what they said, what they think and believe, what they do, how they feel, and their deeper emotional and aspirational needs.

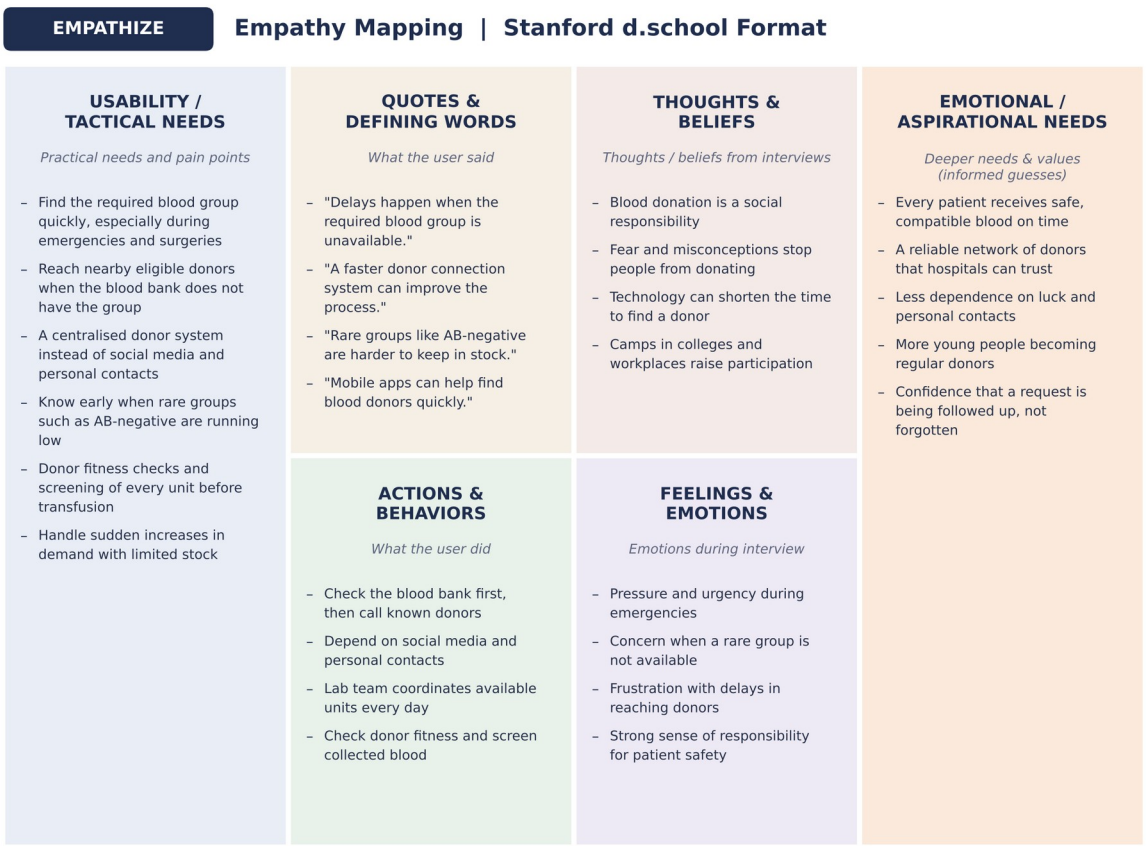


Figure 2.3: Empathy Map (Stanford d.school Format)

2.2 Define

In the Define stage the insights were converted into How Might We (HMW) questions. The team first framed three broad HMW questions from the interviews:

1. How might we help hospitals find compatible blood donors quickly during emergencies?

2. How might we connect nearby eligible blood donors with patients who urgently need blood?

3. How might we encourage more people to become regular blood donors and improve blood availability?

The user needs identified from the interviews are:

- A centralised donor system, so that hospitals do not depend on social media and personal contacts.
- Fast matching of the required blood group with nearby, available and consenting donors.
- Early visibility of blood-bank stock, especially for rare groups such as AB-negative.
- Verification of every emergency request before donors are alerted.
- A way to know whether enough donors have actually agreed, instead of alerting everyone and waiting.
- Clear wording that a donor who agrees is a volunteer; screening and suitability remain with the hospital.

Combining the first two questions with the need to follow up every request, the problem addressed by BloodConnect 2.0 is defined as:

Problem Statement: *How might we make sure enough nearby eligible donors respond to an emergency blood request in time, without alerting every donor at once and while keeping every decision explainable?*

2.3 Ideate

The team wrote the project name and the HMW questions on a chart paper and brainstormed as many ideas as possible without judging them (divergent thinking). Forty ideas were listed, including a blood donor mobile application, a nearby donor finder, blood-group and location-based matching, a rare blood group registry, emergency SOS requests, hospital-verified requests, a blood bank inventory

dashboard, real-time stock updates, low-stock alerts, shortage prediction, a donor eligibility tracker, SMS and WhatsApp alerts, Tamil voice-based search, college, family and community donor networks, a digital donor ID, an AI-powered donor matcher, an emergency donor map and blood camp reminders. The ideas were then narrowed down (convergent thinking) to three concepts, each with the reasons for choosing it, as shown on the right side of the chart in Figure 2.4.

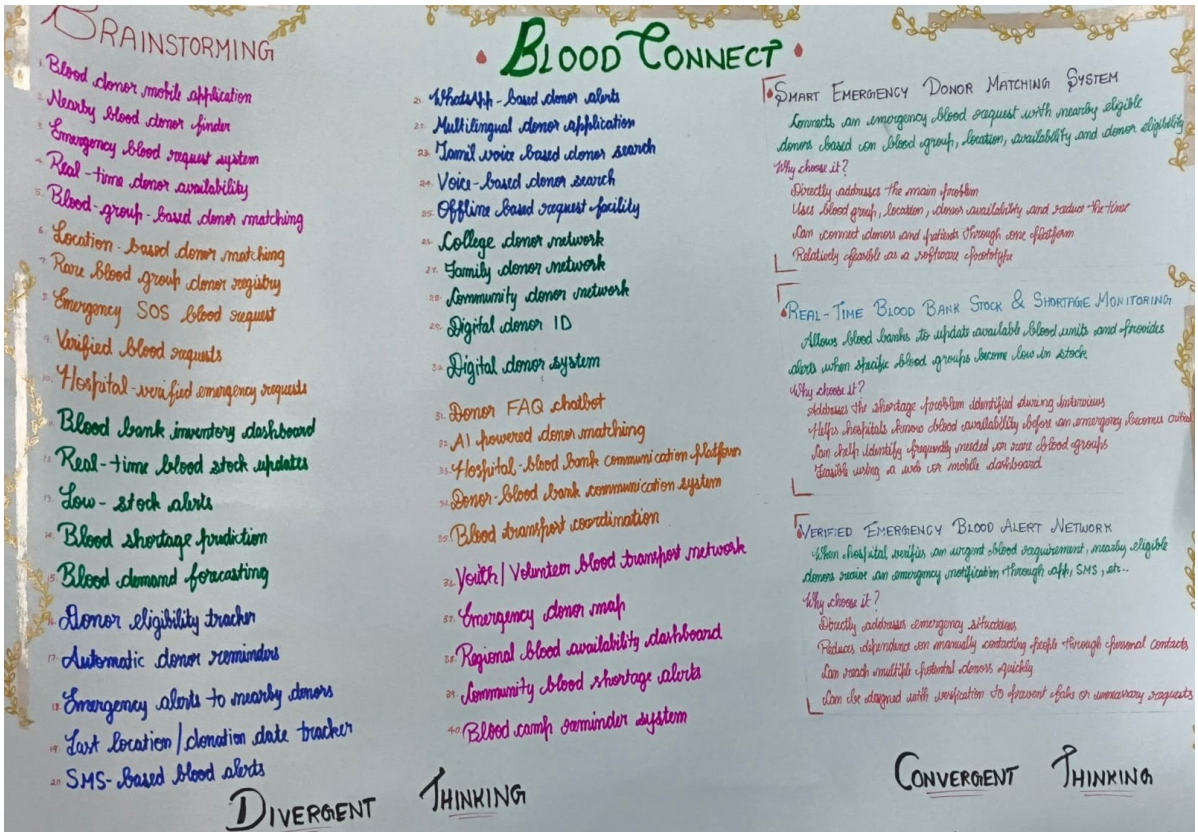


Figure 2.4: Brainstorming Chart: Divergent and Convergent Thinking

The ideas were also organised in a mind map around the central goal of finding blood donors in time (Figure 2.5). The branches cover donor discovery, alerts and communication, blood bank and stock, verification and trust, and community and awareness. The three concepts selected through convergent thinking (highlighted) are:

1. **Smart Emergency Donor Matching System** – connects an emergency request with nearby eligible donors based on blood group, location, availability and donor eligibility. It directly addresses the main problem found in the interviews, reduces search time and is feasible as a software prototype.

2. **Real-Time Blood Bank Stock & Shortage Monitoring** – lets blood banks update available units and raises alerts when specific groups become low, so that shortages, especially of rare groups, are seen before an emergency becomes critical.

3. **Verified Emergency Blood Alert Network** – once a hospital verifies an urgent requirement, nearby eligible donors receive an emergency notification. It reduces dependence on manually contacting people through personal contacts and prevents fake or unnecessary requests.



Figure 2.5: Mind Map and the Three Selected Concepts

The three concepts were combined into a single platform, BloodConnect. While building it, the team saw that the interview pain point “delays in emergencies” is not solved only by alerting donors: someone must also follow up on who answered. This led to BloodConnect 2.0, which adds an Adaptive Emergency Escalation

Engine behind the verified alert network. The features selected for the application are:

- Hospital or administrator verification of every request before any donor is alerted.
- Donor matching on blood group, proximity, verified profile and freshness of declared availability, with a reason for every match.
- Alerts to a small batch of the nearest suitable donors first, with response windows by urgency (10 / 20 / 45 minutes).
- Automatic widening of the search area in four stages (5 km, 10 km, 20 km, whole metro area) only when needed.
- A live count of accepted volunteers and remaining need for every emergency, shown in an Emergency Command Center.
- Blood-bank stock levels and low-stock alerts for every group and component.
- An event timeline, audit log and rule-based decision-support assistant.
- An Emergency Simulator and Strategy Comparison for safe, evidence-based testing.

2.4 Prototype

The selected ideas were developed into a working web application. The backend is written in Python using Django and Django REST Framework with an SQLite database, and the frontend is written in React with Vite and Tailwind CSS. The new engine lives in its own Django app named *escalation* (about 4,900 lines including about 1,350 lines of tests), and no new third-party dependency was added. The administrator portal gained five new pages: Emergency Command Center, Outreach & Escalation, Emergency Simulator, Strategy Comparison and Admin Settings. The screens of the prototype are presented in Chapter 3.

2.5 Test

The prototype was tested at three levels. First, an automated test suite of 116 backend tests checks the decision rules, the engine, the API, the simulator and concurrency. Second, the five simulation scenarios were run against three outreach

strategies with five fixed random seeds each. Third, every page was walked through in a headless browser, including a narrow 390 px mobile width. The prototype was evaluated for the following aspects:

- Correct recording of accepted, declined and no-response alerts.
- Correct recalculation of the remaining need and the next decision.
- No duplicate alerts, even when responses arrive at the same moment.
- Immediate stop of outreach on cancellation or expiry.
- Admin-only access and no leakage of donor contact details.
- Clarity of the Command Center, timeline and explanations.

The test results are given in Sections 3.4 and 3.5.

2.6 System Overview

BloodConnect 2.0 follows a three-tier architecture (Figure 2.6). The React single-page application provides the public pages and the requester, donor and administrator portals. It talks to a Django REST API using token authentication. The API stores data in SQLite. Inside the API, the existing apps (accounts, donors, blood requests, matching, blood bank, notifications, audit and dashboard) call the new escalation engine through small, failure-tolerant hooks, so a fault in the engine can never break a donor’s response.

An important design choice is that the decision rules in *decision.py* are a pure function: they take a snapshot of the case and return a decision, without touching the database, the clock or the user interface. This makes the rules easy to unit-test. The service layer (*service.py*) wraps every change in a database transaction with a row lock, so that repeating a response or an evaluation has no extra effect. The simulator reuses the same engine with a virtual clock.

Every request from the browser carries an authentication token. The API checks the role of the user before any data is returned, so requesters see only their own requests, donors see only the alerts sent to them, and only administrators can open the escalation pages.

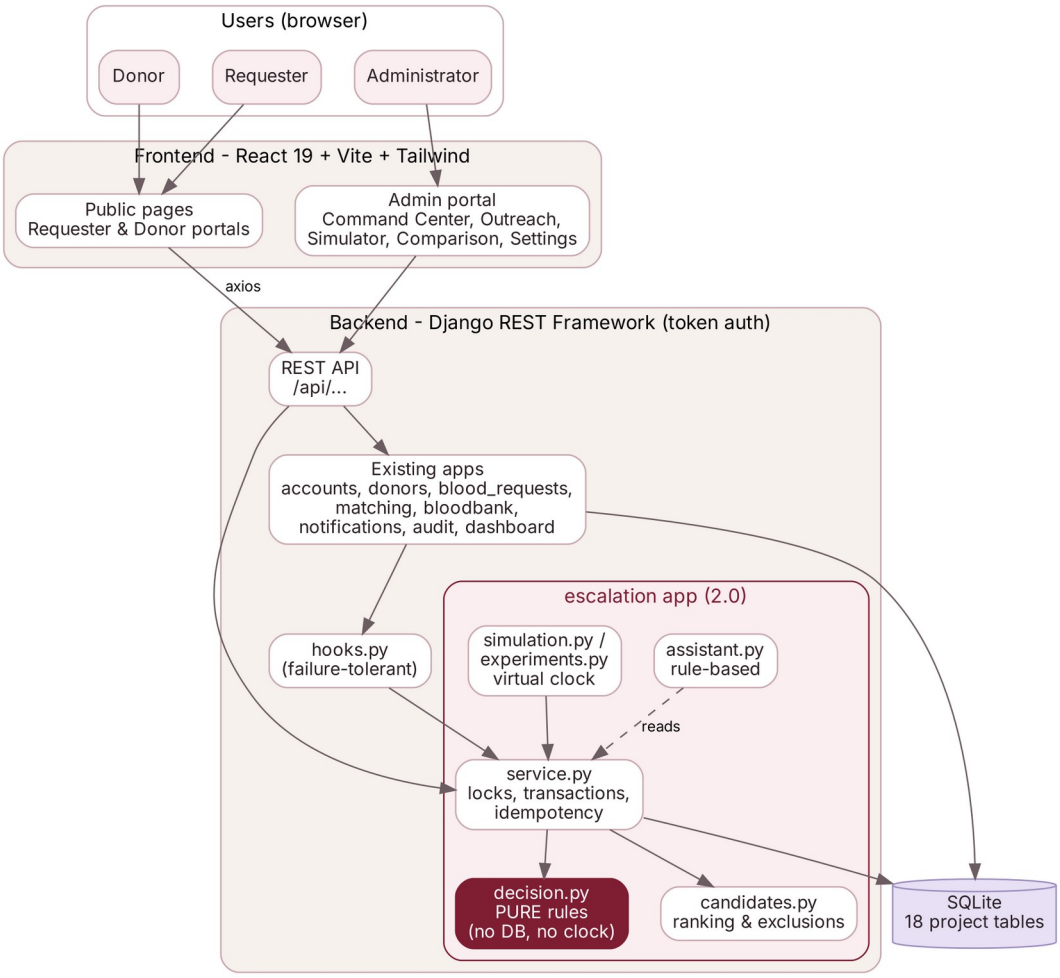


Figure 2.6: System Architecture of BloodConnect 2.0

2.7 User Roles

BloodConnect has three roles. Table 2.3 summarises what each role can do.

Table 2.3: User Roles

Role	Responsibilities
Requester (hospital or family contact)	Creates a blood request with group, component, units, hospital, urgency and required-by time; tracks its status; cancels it when no longer needed.
Donor	Keeps a profile with blood group, locality, availability and consent; receives in-app alerts; accepts or declines.
Administrator	Verifies requests; manages donors, users and blood-bank stock; runs the Command Center, Outreach, Simulator, Strategy Comparison and Settings pages; reviews the audit log.

2.8 System Workflow

The workflow of a real emergency, from request to hospital confirmation, is shown in Figure 2.7. A request starts in the *pending verification* state. Once an administrator approves it, the engine opens an emergency case, ranks donors and

alerts the first batch. Each donor response, each lapsed response window and each administrator action triggers a new evaluation. The loop continues until the volunteers are secured, the request is cancelled or expires, or the engine asks an administrator to intervene.

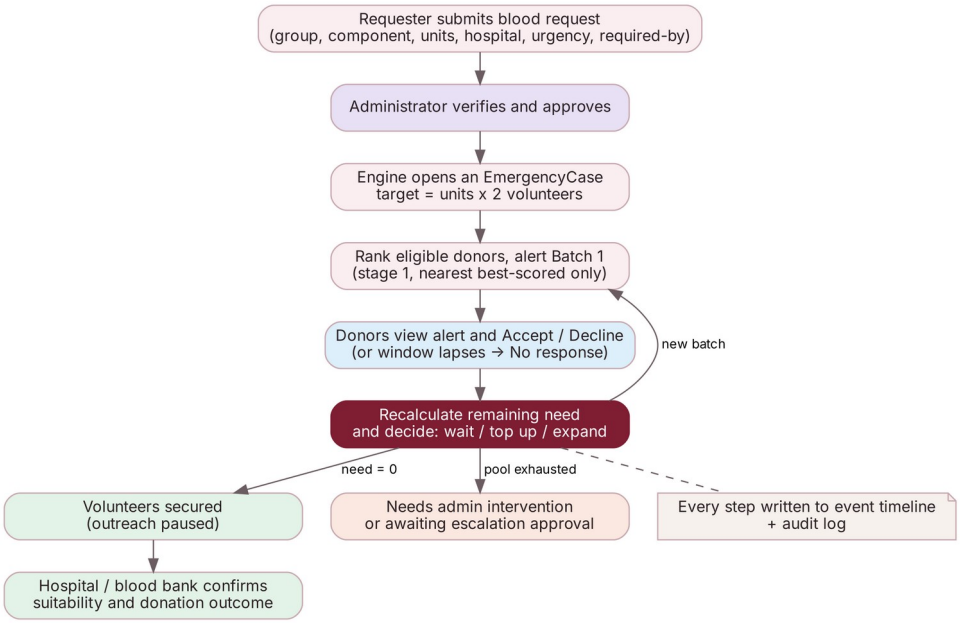


Figure 2.7: System Workflow

2.9 Escalation Algorithm

This section describes the rules used by the engine. All thresholds are stored in an escalation policy that administrators can edit.

2.9.1 Target and Remaining Need

For each request the engine computes *target volunteers = units requested × volunteers per unit* (default 2, as a buffer for hospital deferrals) and *remaining need = max(0, target − accepted volunteers)*. For example, a request for 2 units has a target of 4 volunteers; after two acceptances the remaining need is 2. If the hospital later records that a volunteer did not donate, that volunteer is removed and the need reopens.

2.9.2 Eligibility and Candidate Scoring

A donor is excluded if they have not given consent, have declared themselves unavailable, have an inactive account, are the requester themself, have already declined or accepted this request, have already been contacted, are missing the data

the rules need, or are outside every search stage. The remaining donors are ranked with a transparent score out of 100 (Table 2.4). Past response behaviour is deliberately not used, so that people who were simply busy once are not penalised. When a donor's distance is unknown no value is invented: the same locality gets partial credit and the screen shows "Unknown".

Table 2.4: Candidate Scoring Criteria (default weights)

Criterion	Weight	What it measures
Proximity	35	Distance from the hospital; closer donors score higher.
Blood-group criteria	25	Exact group match scores full; a listed substitute group scores less.
Profile verified by blood bank	20	Whether authorised staff have marked the profile as verified.
Availability freshness	20	How recently the donor declared or updated availability.

2.9.3 Search Stages and Response Windows

The search area grows in cumulative stages (Table 2.5). An alert that receives no answer within the response window for the request's urgency (Critical 10 minutes, Urgent 20 minutes, Standard 45 minutes) is marked *no response*, which means its status is unknown; it is not counted as a refusal.

Table 2.5: Default Search Stages

Stage	Name	Radius	If distance unknown	Max batch
1	Immediate area	5 km	Same locality	8
2	Nearby localities	10 km	Same city	10
3	Greater Chennai	20 km	Same city	14
4	Whole metro area	No limit	Same city	20

2.9.4 Batch Size

The engine estimates how many alerts are needed using an expected acceptance rate that starts from a configured prior and is gradually replaced by what is actually observed:

expected acceptance = (accepted + prior × prior weight) / (answered + prior weight)

With the default prior of 0.35 worth 4 virtual alerts and no answers yet, the expected acceptance is 0.35. For a need of 4 volunteers the first batch would be ceil(4 / 0.35) = 12, which is capped by the stage-1 maximum of 8. If fewer donors are eligible in stage 1, only those are alerted.

2.9.5 Decision Cycle

After every event the engine evaluates the case under a row lock and applies the rules below in priority order (Figure 2.8):

1. If the request is cancelled, fulfilled or past its required time plus a grace period, close the case and withdraw pending alerts.
2. Mark alerts older than the response window as no response.
3. If the remaining need is zero, mark the case as volunteers secured and pause outreach.
4. If no batch has been sent yet, send the first batch from stage 1.
5. If pending alerts are expected to cover the need, or the last batch was too recent, wait.
6. Otherwise, top up in the current stage, sized by the shortfall.
7. If the current stage has nobody left and earlier alerts are answered or lapsed, expand to the next stage (or wait for an administrator’s approval if auto-escalation is off).
8. If nobody is left or the batch limit is reached, flag the case as needing administrator intervention and notify administrators once.

Because the rules are checked in a fixed order, the first rule that applies decides the outcome. A cancelled request is therefore closed even if alerts are still pending, and a case whose need is already met never sends another batch.

The same decision function is used for real emergencies and for simulations. In a simulation the time comes from a virtual clock, so a full two-hour emergency can be replayed in a few seconds with exactly the same rules.

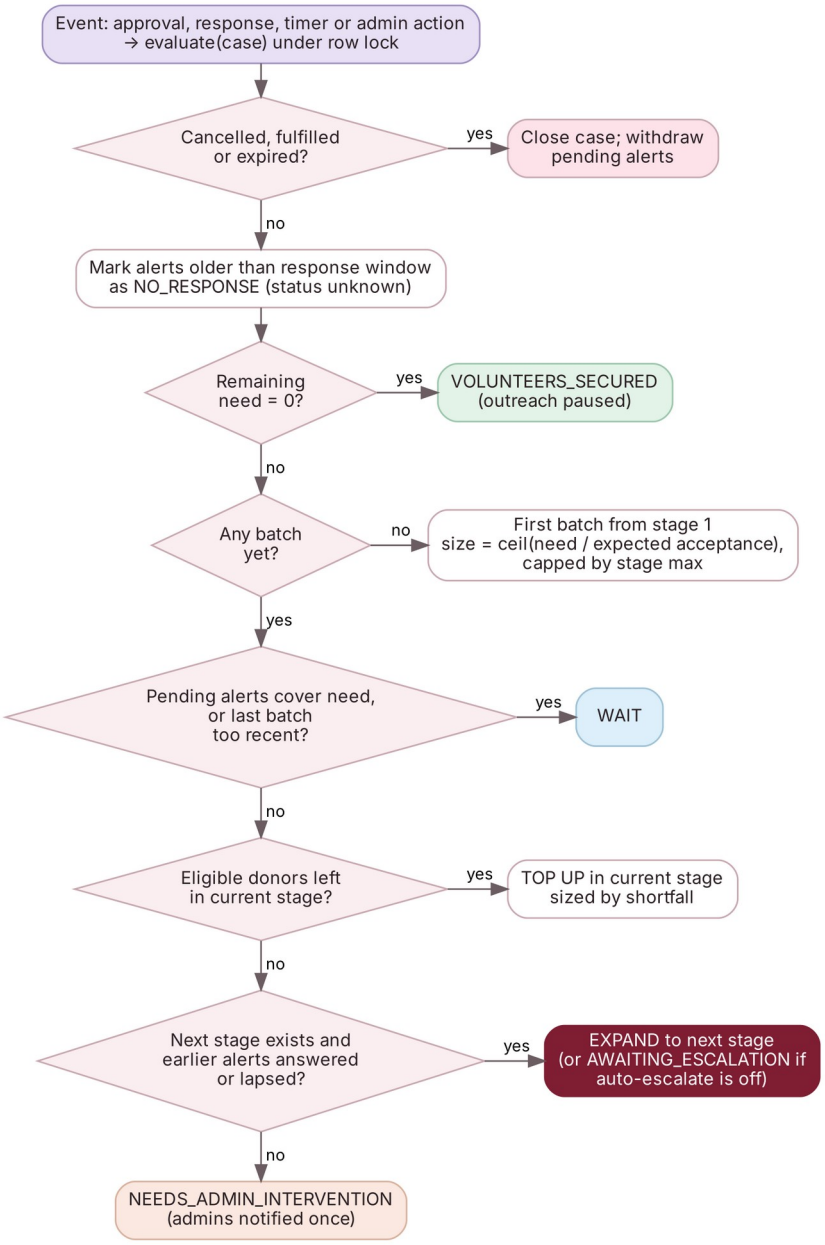


Figure 2.8: Decision Cycle of the Escalation Engine

2.10 Main Modules

The application is divided into the modules described in Table 2.6.

Table 2.6: Main Modules of BloodConnect 2.0

Module	Description
Public site	Landing, How It Works, Blood Awareness, About, Contact, Login and Register pages.
Requester portal	Dashboard, create request, my requests, request detail, matches and notifications.
Donor portal	Dashboard with alerts, profile and availability, relevant requests, alert detail with accept/decline, donation history.
Verification and requests	Administrator verification queue and request management; approval starts the engine.

Module	Description
Emergency Command Center	All active emergencies with status, stage, accepted volunteers, remaining need and a city map.
Outreach & Escalation	Per-case metrics, outreach pipeline, required action, candidate table with score breakdown, timeline, map and decision support.
Emergency Simulator	Five fictional scenarios with play, step, pause, reset and run-to-end controls.
Strategy Comparison	Runs one scenario through Broadcast, Fixed and Adaptive strategies over several seeds and shows tables and charts.
Admin Settings	Edits volunteers per unit, response windows, stages, weights and other policy values.
Blood-bank stock	Stock levels per bank, group and component, low-stock alerts and stock transactions.
Notifications and audit	In-app notifications and an audit log of administrator actions.

2.11 Database Design

SQLite is used as the database. The project defines 18 tables across its Django apps, in addition to Django's built-in tables. The main tables are listed in Table 2.7 and their relationships are shown in Figure 2.9. Table 2.7 also gives the number of rows in the seeded database on 29 October 2026.

Table 2.7: Main Database Tables

Table	Main data stored	Rows
User	E-mail (login), role, phone, city, locality.	42
DonorProfile	Blood group, locality, approximate coordinates, availability, consent, verification.	32
BloodRequest	Reference, group, component, units, hospital, urgency, required-by, status.	15
DonorMatch	Original alert row per request and donor with its alert status.	107
Notification	In-app notification with type, urgency and read status.	158
BloodBank / BloodStock	Blood banks and units available per group and component.	11 / 88
EmergencyCase	One per real request or simulation: status, stage, target, remaining need.	5
OutreachBatch	Each batch of alerts with its stage, trigger and reason.	5
OutreachAttempt	One alert to one donor: status, rank score, score breakdown, explanation.	83
EscalationEvent	Append-only timeline of every decision with its reason.	30
EscalationPolicy	Active policy as validated JSON.	1

Table	Main data stored	Rows
SimulationRun / SimulatedDonor / Experiment	Fictional runs, donors and comparison results (created on demand).	0

Four database constraints protect the integrity of the escalation data:

- A case is either real (linked to a request) or simulated (linked to a simulation run), never both.
- Each outreach attempt refers to exactly one kind of donor, real or fictional.
- A donor can be contacted only once per case and stage, which prevents duplicate alerts.
- Batch sequence numbers are unique within a case.

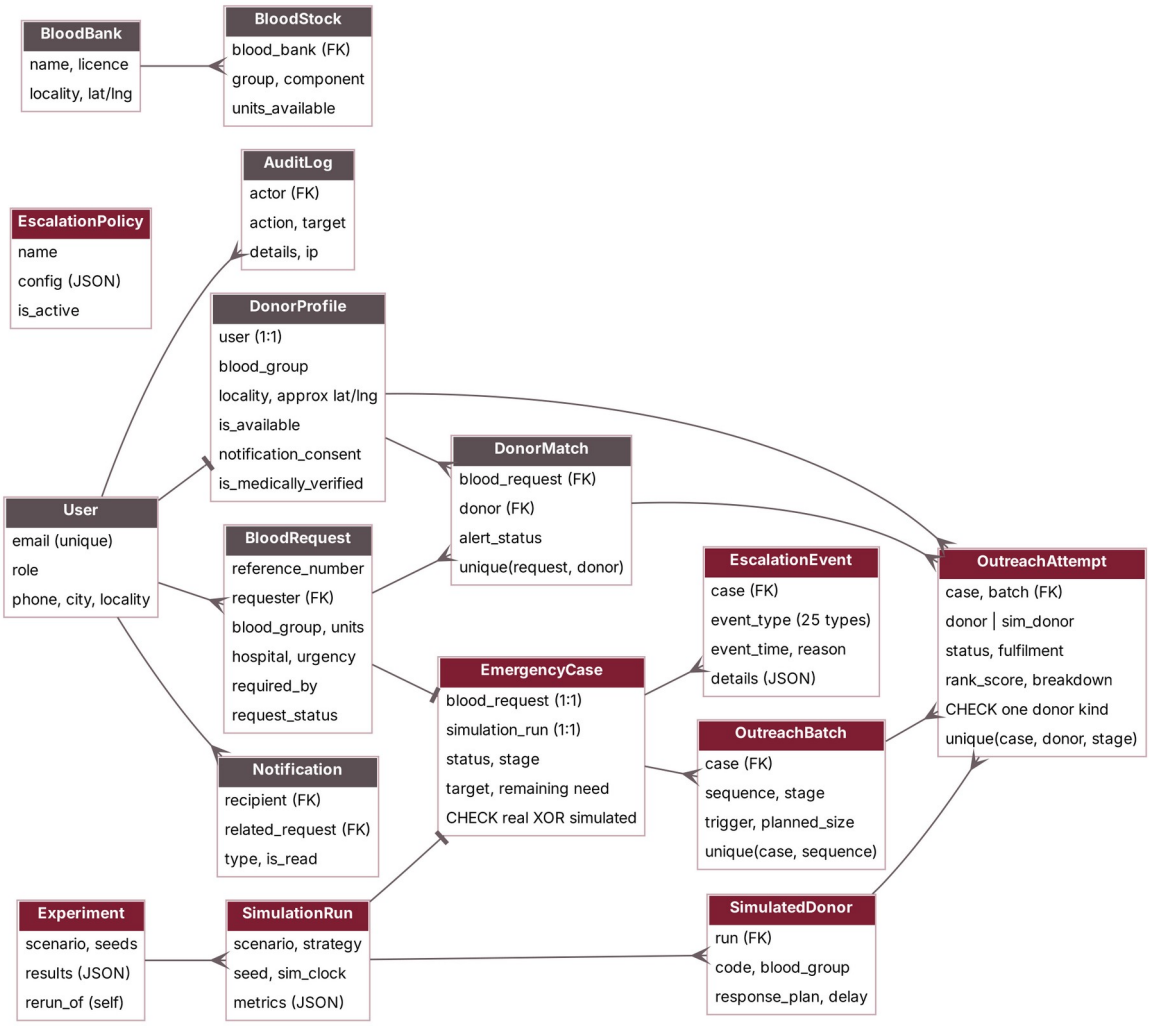


Figure 2.9: Entity-Relationship Diagram

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Implementation Overview

BloodConnect 2.0 was implemented as a web application. The technologies used are listed in Table 3.1. The interface uses a burgundy and warm-neutral healthcare palette, rounded cards, a dark sidebar for the portals and a clinical safety notice on every coordination screen.

Table 3.1: Technology Stack

Layer	Technology
Backend	Python, Django 6.1, Django REST Framework, django-cors-headers, python-dotenv
Authentication	DRF token authentication; custom User model with e-mail login and three roles
Database	SQLite (transaction mode IMMEDIATE); PostgreSQL possible through configuration
Frontend	React 19, Vite 8, Tailwind CSS v4, React Router, Axios
Maps and charts	React-Leaflet with OpenStreetMap tiles; Recharts
Icons	Lucide React
Testing	Django TestCase and TransactionTestCase; headless-browser walkthrough

3.2 User Interface

Figure 3.1 shows the public landing page and the sign-in page. The landing page explains the coordination flow and the sign-in page leads each role to its own portal.

New users can register as a requester or as a donor from the same page. Donor registration collects the blood group, locality, availability and consent to receive emergency alerts.

The landing page also links to the How It Works, Blood Awareness, About and Contact pages, and shows a clinical safety notice stating that the platform only matches potential donors and that transfusion compatibility is confirmed by the blood bank.

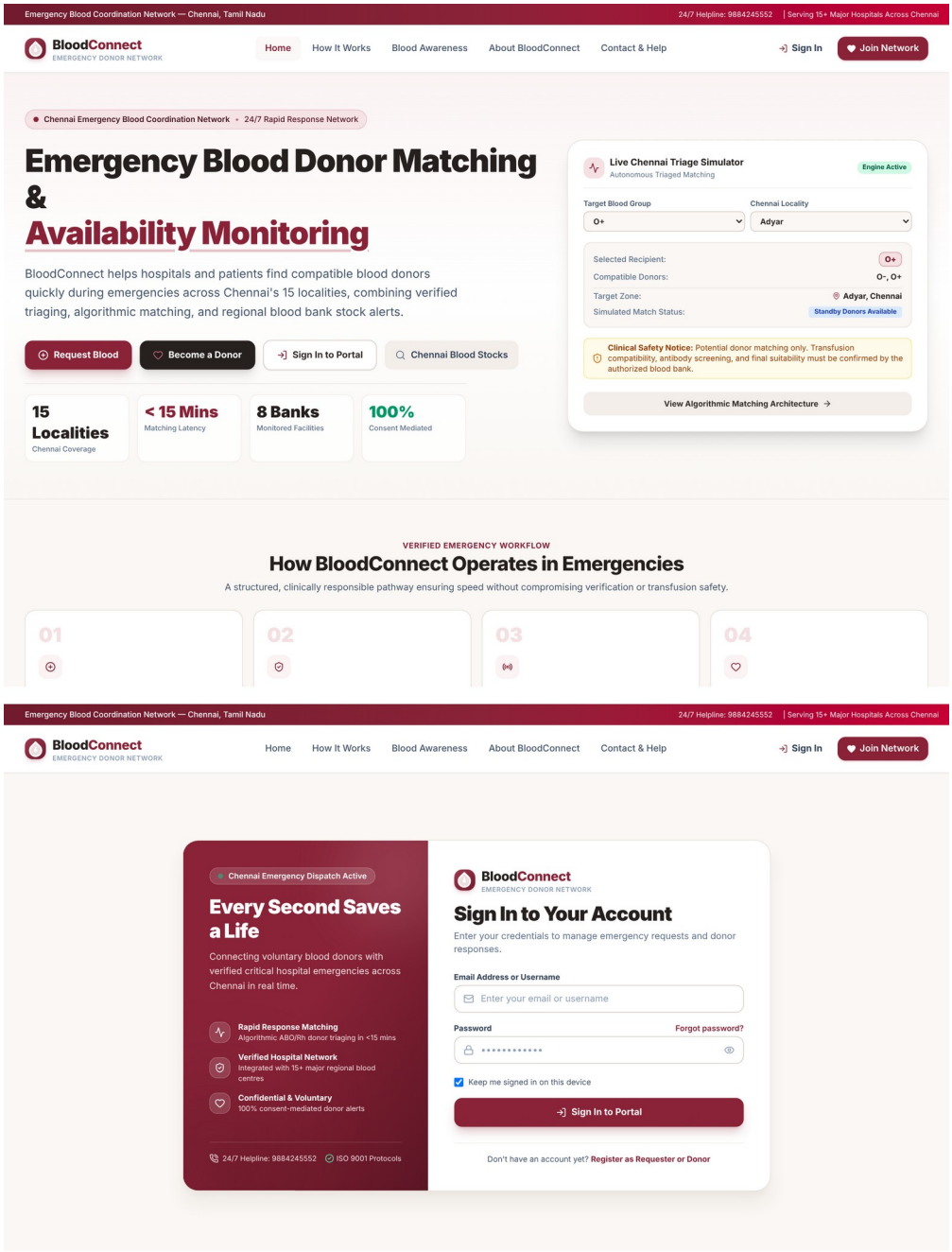


Figure 3.1: Landing Page and Sign-in Page

Figure 3.2 shows the requester's Submit Emergency Blood Request form. The requester chooses the blood group, component, units and urgency, gives the hospital details and the required-by time, and confirms that the request is genuine. Every new request waits for administrator verification before any donor is alerted. After submission the requester can follow the request from the My Requests page, which shows its verification status, the number of volunteers who have accepted and any updates from the administrator.

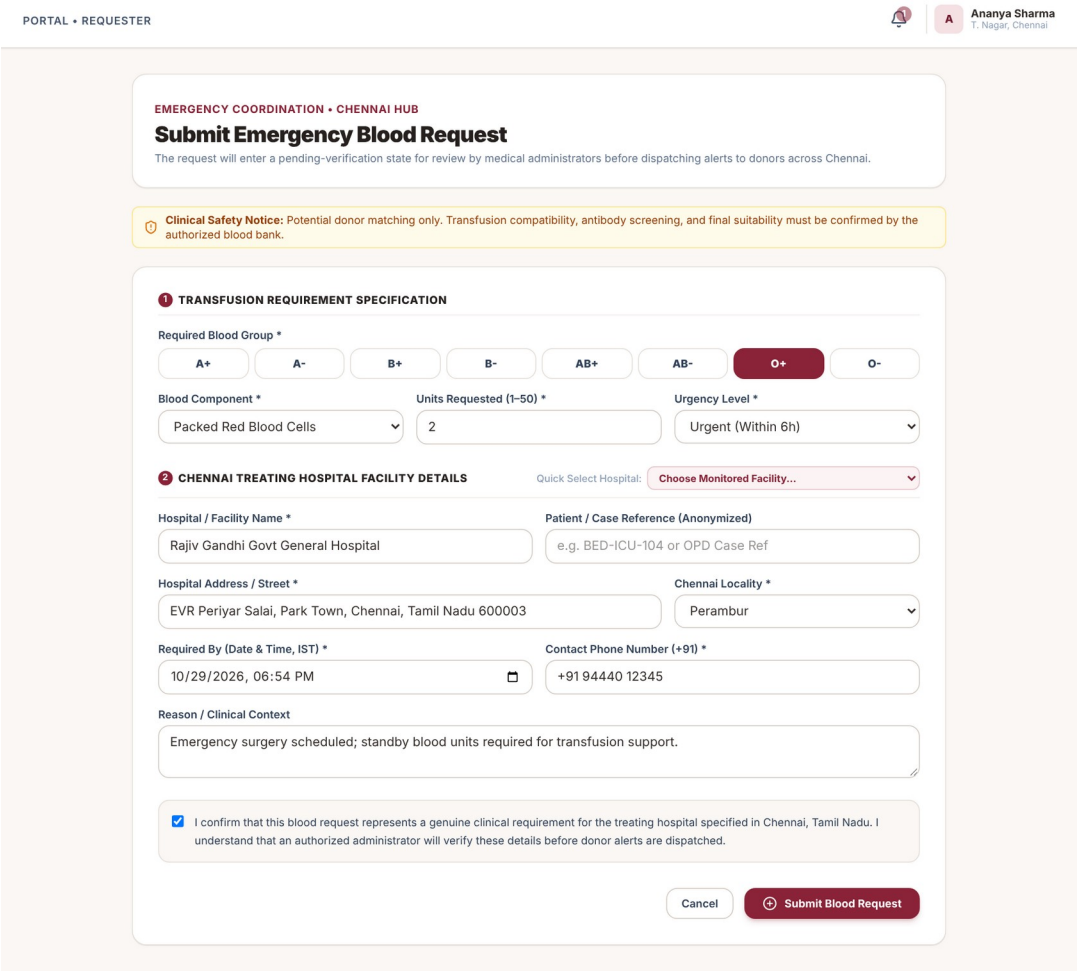


Figure 3.2: Submit Emergency Blood Request (Requester)

Figure 3.3 shows the Volunteer Donor Portal. The donor can switch availability on or off and sees the emergency alerts sent to them, each with a Respond button.

When a donor accepts, the response is recorded immediately and the remaining need of the emergency is recalculated. When a donor declines, the engine stops contacting that donor for the same request and, if needed, looks for the next suitable donor.

Each alert card shows the request reference, the blood group, the hospital and the urgency. Opening an alert records that the donor has seen it, which is the only evidence of delivery the system has, since alerts are in-app notifications.

The availability switch at the top of the portal lets a donor stop receiving alerts at any time, for example after a recent donation or during illness. Donors who switch it off are excluded from every new search.

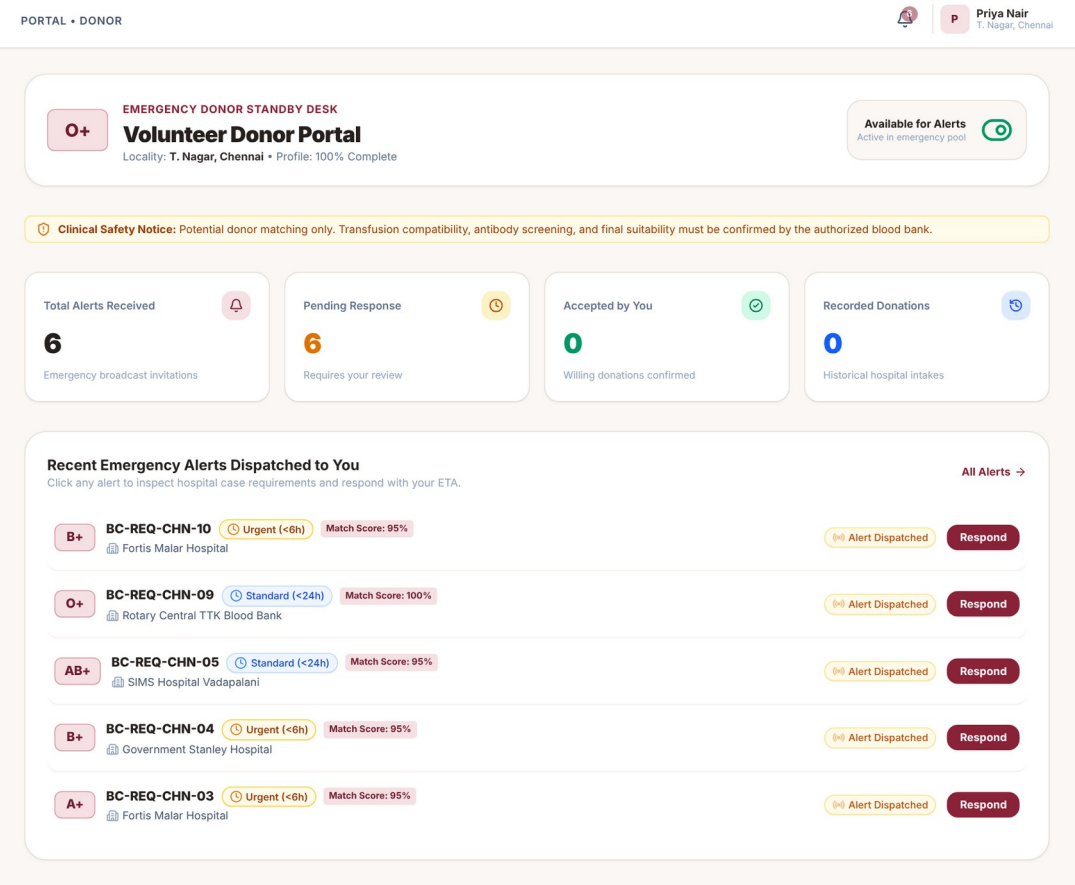


Figure 3.3: Volunteer Donor Portal

Figure 3.4 shows the Administrator Command & Control Portal with registered donors, pending verifications, active cases, low-stock warnings, blood-stock reserves by group and the distribution of donors across Chennai localities.

From the sidebar the administrator can reach the emergency coordination pages, the verification queue, request and donor management, blood-bank stock, platform alerts and the audit log, so every task is one click away.

The cards along the top give the number of registered donors, pending verifications, active cases, accepted volunteers, fulfilled requests and total units in reserve. The lists at the bottom show recent requests with their status and the latest donor responses.

Low-stock warnings are raised when the units available for a group and component fall below the threshold configured for that blood bank.

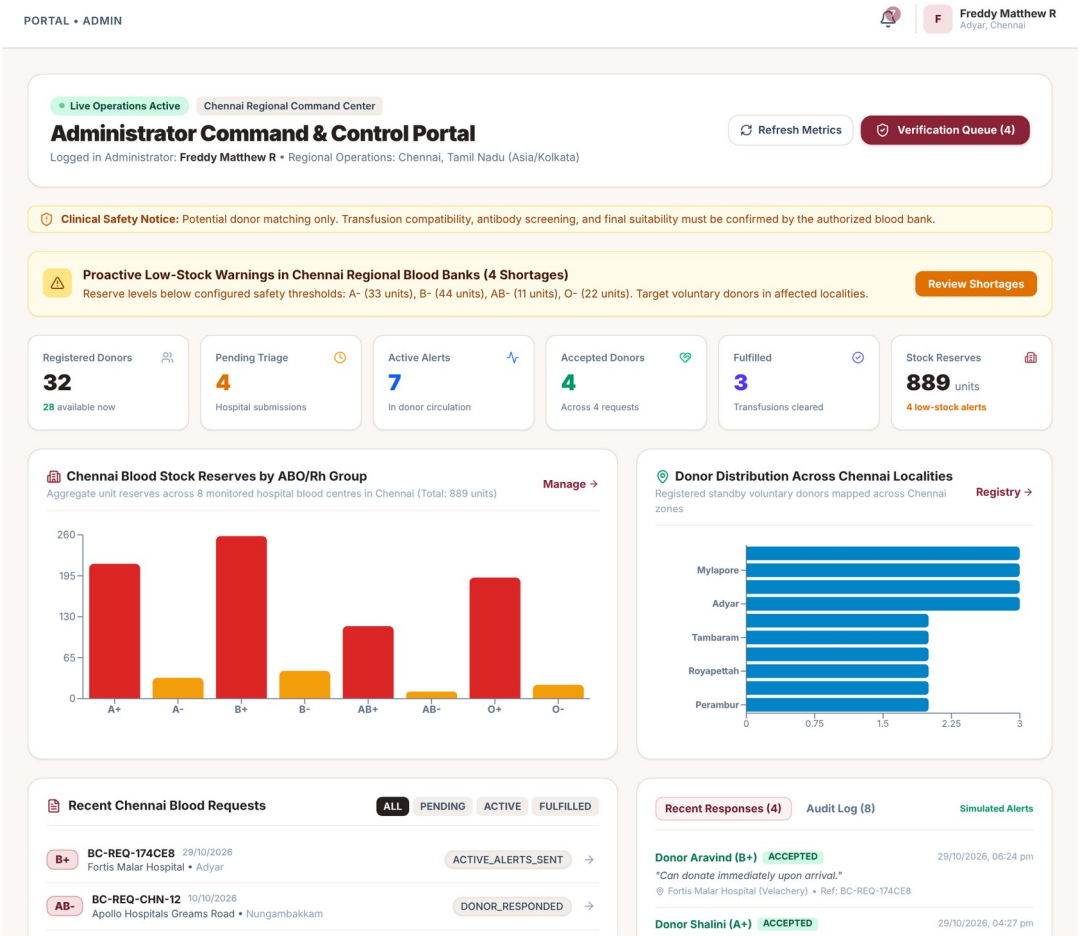


Figure 3.4: Administrator Dashboard

Figure 3.5 shows the Emergency Command Center. Each active emergency is listed with its group, units, hospital, urgency, status, potential candidates, outreach attempts, volunteers accepted and remaining need. The note under the table makes clear that accepted volunteers are not confirmed donations. A map shows where the emergencies are.

Clicking any row opens the Outreach & Escalation page of that emergency, where the administrator can see each donor contacted and the reason for every decision.

The Command Center can re-evaluate all active emergencies every 30 seconds while it is open, and it has an Evaluate all now button. Administrators can also create a new emergency directly from this page.

Requests that were approved before the escalation engine existed are listed separately and can be brought under coordination with one click; their earlier alerts are imported and never repeated.

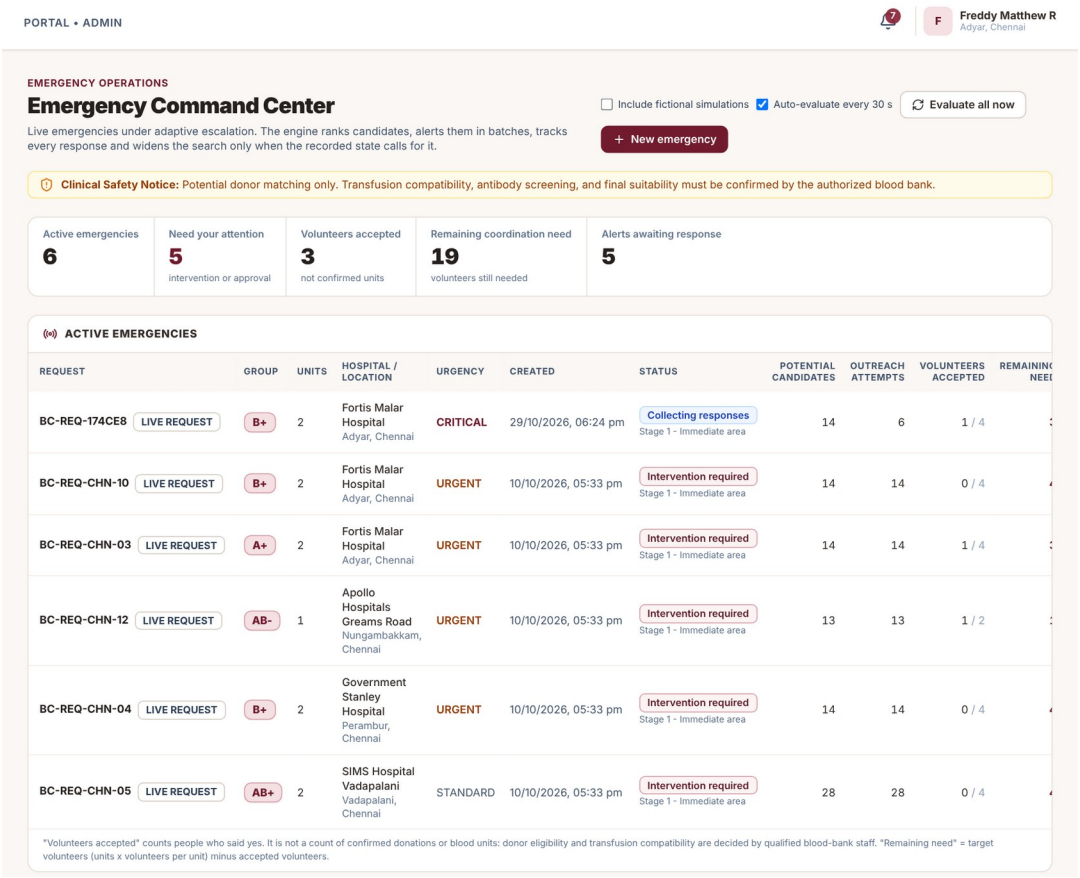


Figure 3.5: Emergency Command Center

Figure 3.6 shows the Outreach & Escalation page for a demonstration request of 2 units of B+ at Fortis Malar Hospital, Adyar (Critical). The target is 4 volunteers; after one acceptance the remaining need is 3. The pipeline shows the stage reached and the Required Action panel explains why the engine is currently holding.

The outreach pipeline shows seven steps from the initial search to resolution or cancellation. Steps that are complete are marked in green and the current step is highlighted, so the state of the emergency can be read at a glance.

The buttons at the top let an administrator run the engine immediately, approve the next batch when automatic escalation is switched off, mark the case as reviewed, resolve it or cancel the request. Every one of these actions is written to the audit log.

The metric cards keep the vocabulary strict: accepted volunteers are shown as “not confirmed units”, alerts without an answer are shown as “status unknown”, and donations are counted only when confirmed by the hospital.

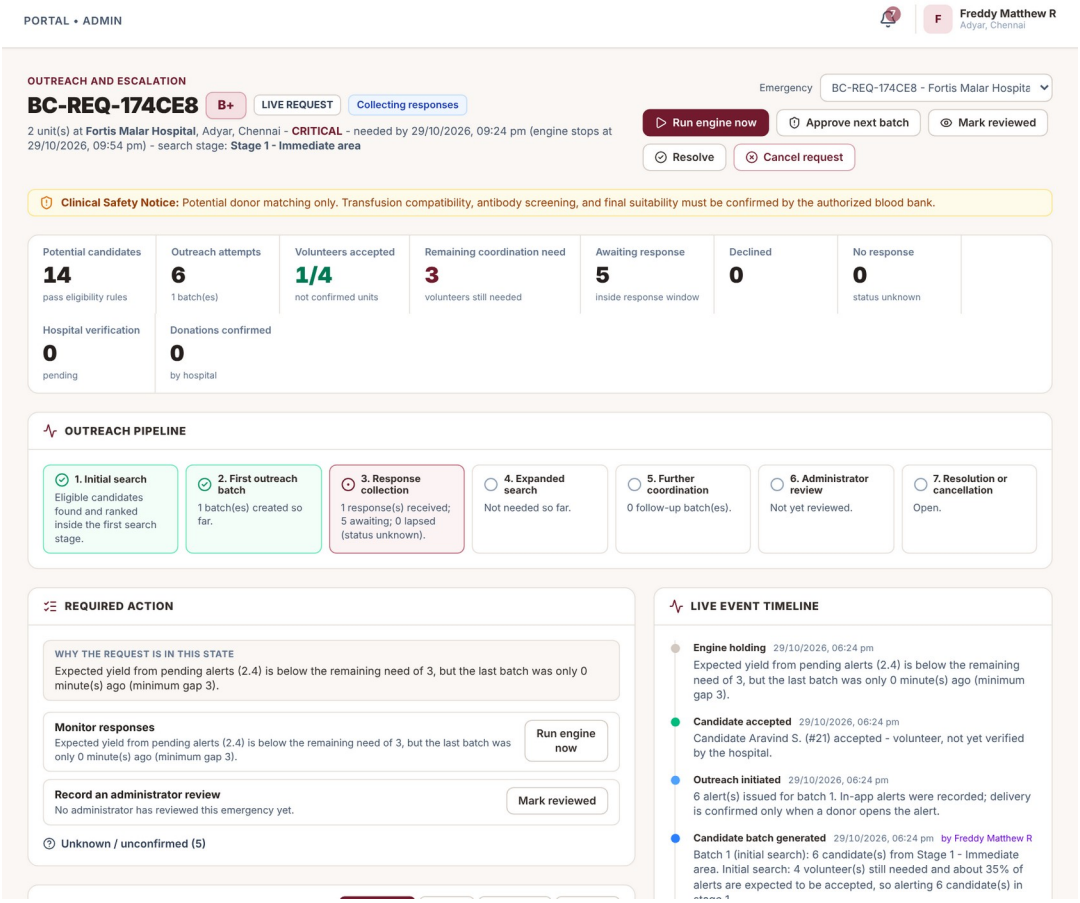


Figure 3.6: Outreach & Escalation: Metrics, Pipeline and Required Action

Figure 3.7 shows the candidate table with the first candidate expanded. The score of 91.8 is broken down into proximity 26.78/35, blood-group criteria 25/25, verified profile 20/20 and availability freshness 20/20, with a plain-language reason. On the right, the decision-support assistant answers “Why did the system prioritise these candidates?” using only recorded data, listing known facts, unknowns and the rules applied.

The decision-support panel offers ready questions such as why the request is still unresolved, why the search was or was not expanded, what information is missing and what actions remain. Each answer is split into known facts, unknowns, rules applied and suggestions.

The candidate table can be filtered to show all considered donors, only the alerted donors, the donors who are next in line, or the donors who were excluded along with the reason for each exclusion. Contact details are never shown.

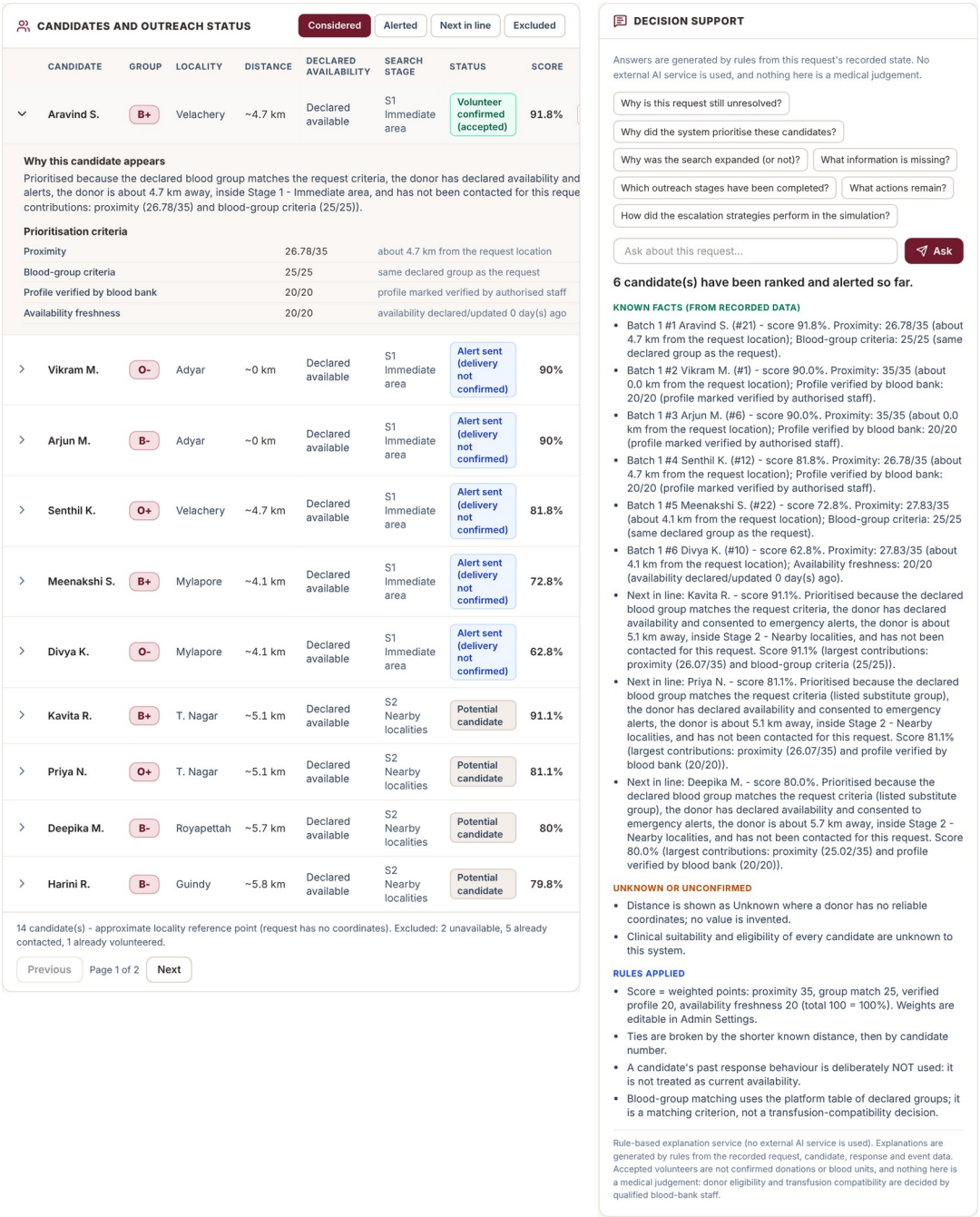


Figure 3.7: Candidate Score Breakdown and Decision Support

Figure 3.8 shows the live event timeline, where every decision is recorded with its reason, and the emergency map, with the hospital, candidates, alerts and the dashed rings of the search stages. Donor positions are rounded to about 1 km and contact details are never shown.

The map draws dashed rings for the search stages around the hospital, and colours each donor marker by status: potential candidate, alert sent, volunteer, declined or no response.

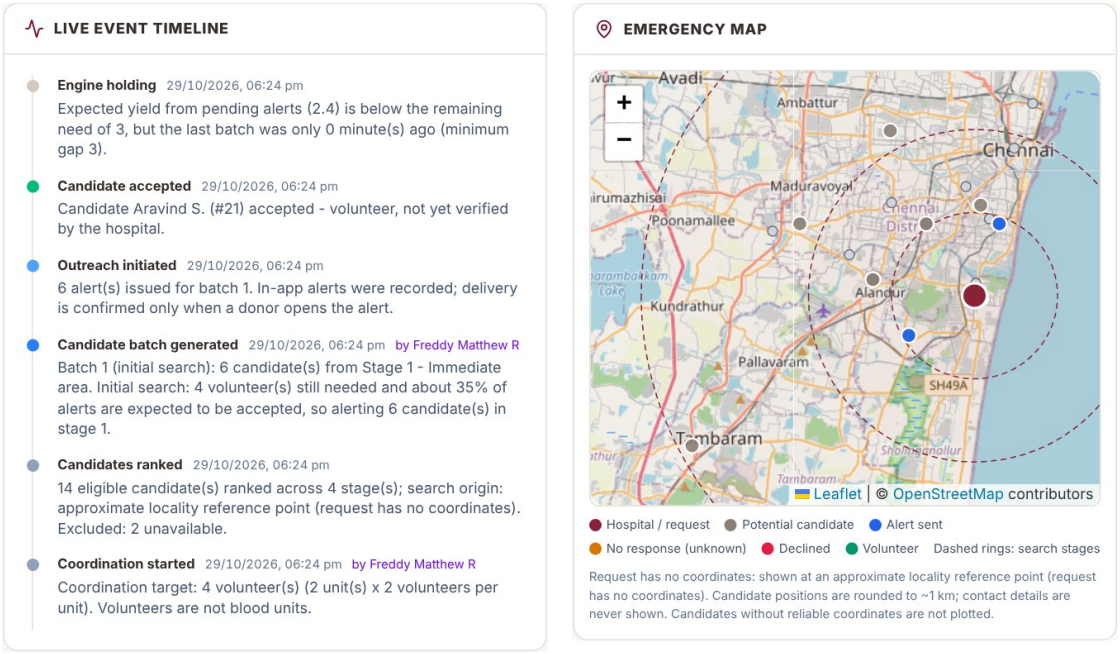


Figure 3.8: Live Event Timeline and Emergency Map

Figure 3.9 shows the Emergency Response Simulator running the “No response in the initial area” scenario with fictional donors. At simulated minute 12, eight alerts in the first stage have lapsed with no response (status unknown) and the engine has sent top-up batches. A banner states that all data is fictional and nothing is sent.

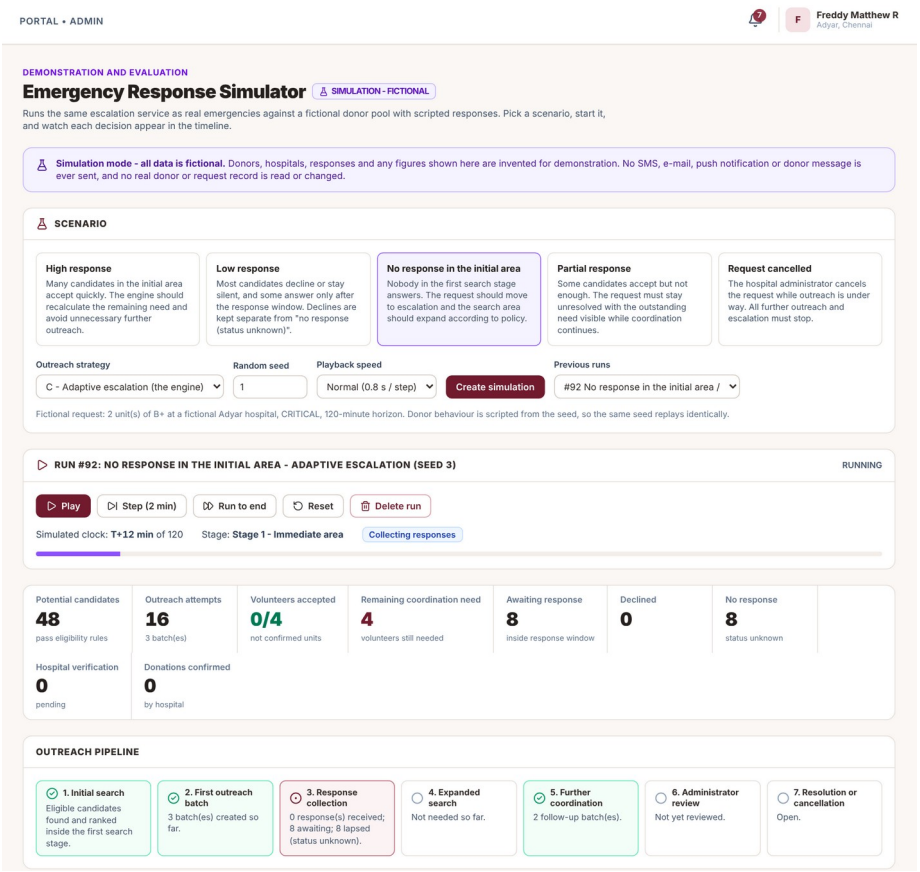


Figure 3.9: Emergency Response Simulator

Figure 3.10 shows the Strategy Comparison page for the high-response scenario over seeds 1 to 5, with a metric table and charts for the three strategies.

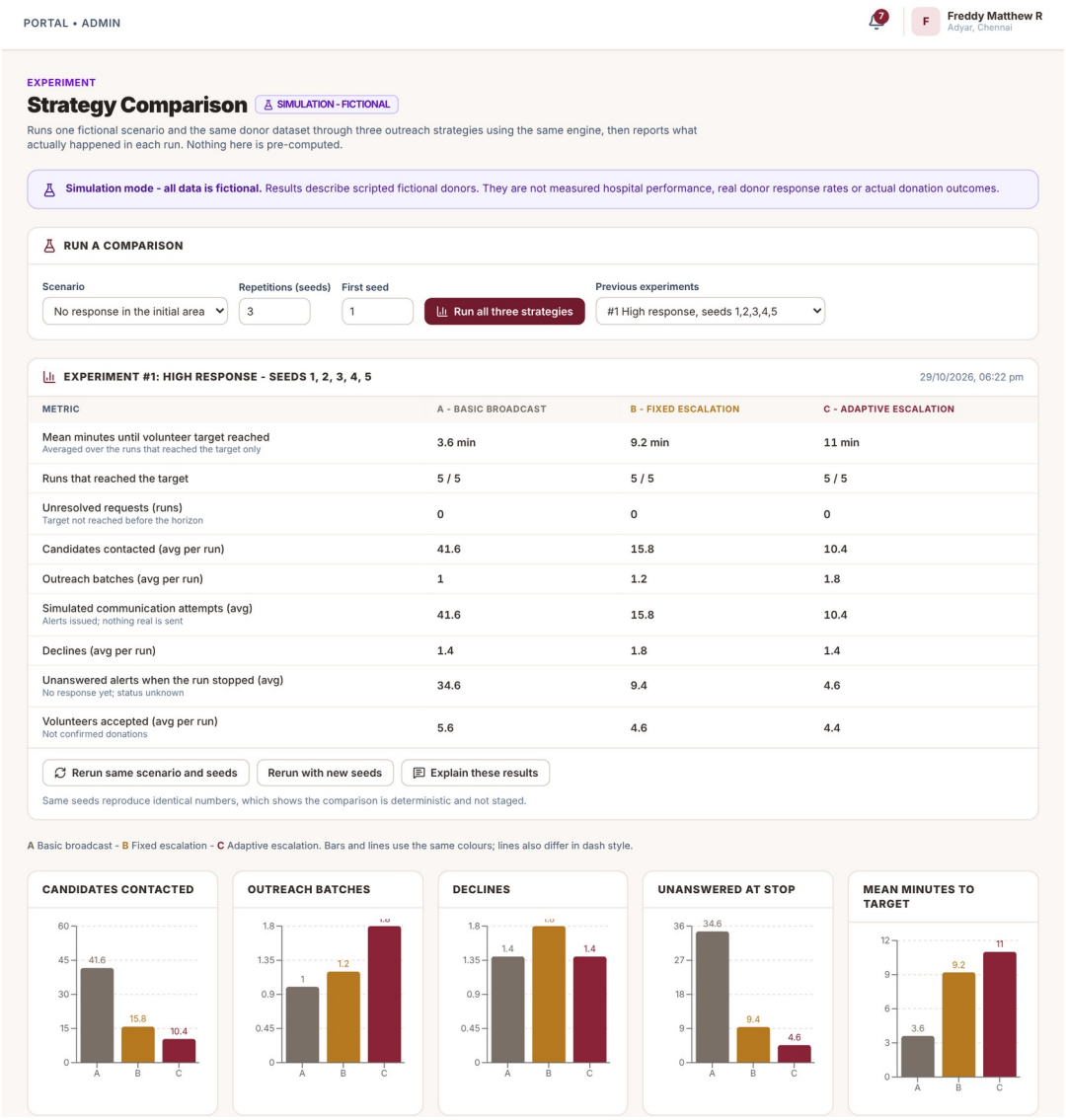


Figure 3.10: Strategy Comparison (High Response Scenario)

Figure 3.11 shows the Admin Settings page, where the volunteers per unit, assumed acceptance rate, response windows, search stages and ranking weights can be changed. Changes are validated and written to the audit log.

Changes take effect for new decisions, and each simulation stores a copy of the policy it used, so results from different policy settings can be compared fairly.

Every value is validated before it is saved: for example, each ranking weight must be between 0 and 100 with at least one above zero, and a later search stage cannot have a smaller radius than an earlier one. A Reset to defaults button restores the original policy.

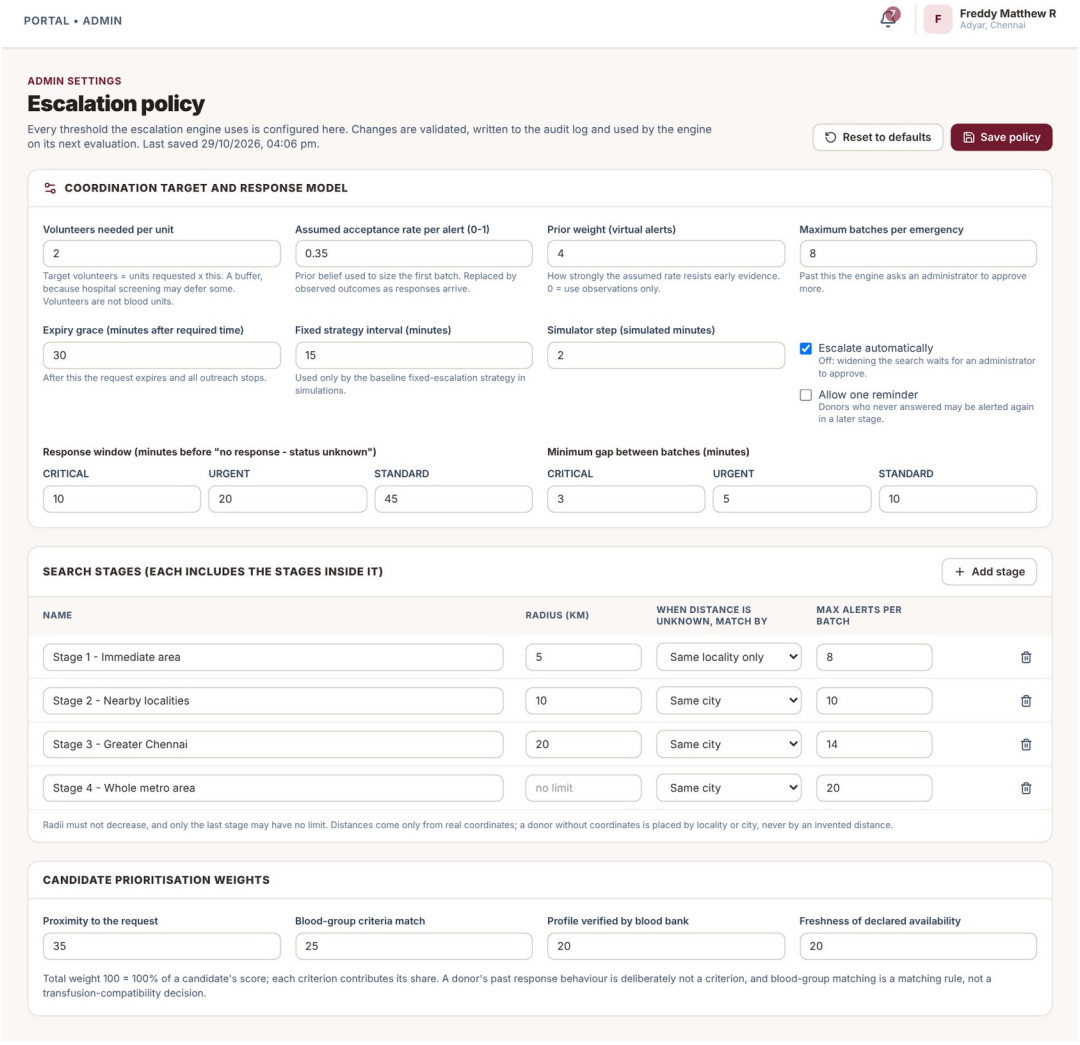


Figure 3.11: Escalation Policy Settings

3.3 Working Procedure

1. The requester signs in and submits an emergency blood request.
2. An administrator reviews the request in the Verification Queue and approves it.
3. The engine opens an emergency case, sets the volunteer target and ranks eligible donors.
4. The first batch of the nearest, best-ranked donors receives in-app alerts.
5. Donors open the alert and accept or decline; unanswered alerts lapse after the response window.
6. After every event the engine recalculates the remaining need and decides to wait, top up, expand or ask for help.
7. Administrators follow progress in the Command Center and the Outreach page and can ask the assistant why.

8. When the need is met, outreach pauses; the hospital or blood bank confirms suitability and the donation outcome.

9. If the request is cancelled or expires, pending alerts are withdrawn and no further outreach is created.

3.4 Evaluation and Testing

The backend test suite was run with *python manage.py test*. All 116 tests passed ("Ran 116 tests ... OK"). Table 3.2 lists the test files and what each proves.

Table 3.2: Automated Test Suite

Test file	Tests	What it checks
escalation/test_decision	15	Pure decision rules without a database.
escalation/test_engine	34	Case creation, eligibility and consent, accept / decline / no response, need recalculation, escalation and approval mode, cancel and expiry.
escalation/test_api	22	Admin-only access, payloads without contact details, audit logging.
escalation/test_simulation	27	Simulation isolation, reproducible results by seed, metrics recomputed from stored rows, assistant output.
escalation/test_concurrency	3	Parallel threads sending responses and evaluations; exactly one is applied.
Existing apps (accounts, requests, blood bank)	15	Register → request → approve → respond flow and stock rules.
Total	116	All passed.

Functional testing was also carried out through the user interface. Table 3.3 summarises the main test cases.

Table 3.3: Functional Test Cases and Results

S.No.	Test Case	Expected Result	Result
1	Approve a 2-unit Critical request	Case opens with target 4; first batch alerts stage-1 donors only	Pass
2	Record one acceptance	Remaining need drops from 4 to 3; event added to the timeline	Pass
3	Let an alert pass its response window	Alert shows "no response (status unknown)", not declined	Pass
4	Exhaust stage 1 with no acceptances	Search expands to stage 2 with a recorded reason	Pass
5	Respond twice to the same alert at once	Only one response is applied; no duplicate alert	Pass
6	Cancel an active request	Pending alerts withdrawn; no new batches	Pass

S.No.	Test Case	Expected Result	Result
7	Open escalation pages as a non-admin	Access refused	Pass
8	Rerun an experiment with the same seeds	Identical numbers are produced	Pass

In addition, the frontend production build (*vite build*) succeeds, and a headless-browser walkthrough of all pages showed no JavaScript or API errors and no horizontal overflow at a 390 px mobile width. The frontend has no automated test suite.

3.5 Simulation Results

The three strategies were compared on the same fictional donor dataset and the same scripted donor behaviour: **A – Basic broadcast** alerts the whole eligible pool once (how version 1 behaved); **B – Fixed escalation** opens each stage on a fixed 15-minute timer and alerts the whole stage; **C – Adaptive** is the engine described in Section 2.9. Each scenario was run with seeds 1 to 5 and the values in Table 3.4 are means over the five runs.

Table 3.4: Strategy Comparison (mean of 5 seeded runs, fictional donors)

Scenario	Strategy	Donors contacted	Batches	Accepted	Target reached	Minutes to target
High response	A Broadcast	41.6	1	5.6	5/5	3.6
	B Fixed	15.8	1.2	4.6	5/5	9.2
	C Adaptive	10.4	1.8	4.4	5/5	11
Partial response	A Broadcast	41.2	1	4.8	5/5	10.8
	B Fixed	24.4	1.6	4.6	5/5	18.2
	C Adaptive	22.2	3.6	4.4	5/5	21
No response (initial area)	A Broadcast	42.8	1	5	5/5	6.2
	B Fixed	32.2	2	4.6	5/5	24.4
	C Adaptive	31.2	4.4	4.4	5/5	27.2
Low response	A Broadcast	37.6	1	3.4	3/5	18.7
	B Fixed	35.8	2.8	3.4	3/5	34.3
	C Adaptive	35.4	5.4	3.4	3/5	35
Request cancelled	A Broadcast	40.2	1	2	1/5	5
	B Fixed	10.2	1	0.4	0/5	not reached
	C Adaptive	9.4	1.4	0.4	0/5	not reached

Figure 3.12 plots the number of donors contacted and the time to reach the volunteer target for every scenario.

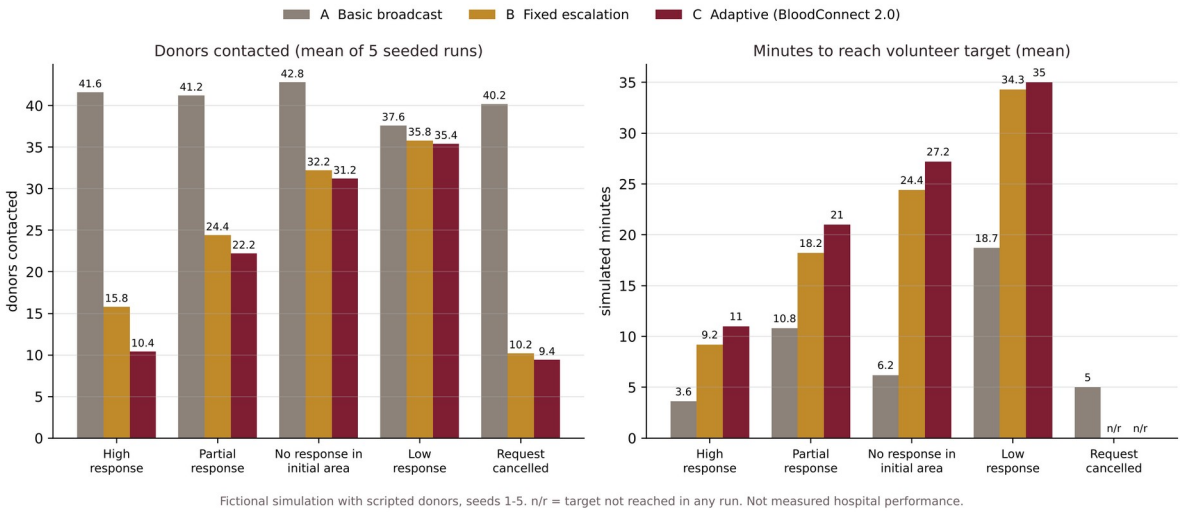


Figure 3.12: Donors Contacted and Time to Target by Strategy

These figures come from a fictional simulation with scripted donor behaviour. They are not measured hospital performance, real donor response rates or actual donation outcomes.

3.6 Discussion

- **Broadcast is the fastest.** In every scenario where the target was reached, alerting everyone at once reached it first (for example 3.6 minutes against 11 minutes for Adaptive in the high-response scenario).
- **Adaptive contacts far fewer donors when donors respond.** In the high-response scenario it contacted about 10 donors against about 42 for Broadcast, a cut of about 75 %; in the partial-response scenario about 22 against about 41.
- **When responses are scarce the strategies converge.** In the low-response and no-response-in-initial-area scenarios the search area has to widen anyway, so all three strategies contact a similar number of people and Adaptive is not faster.
- **Cancellation stops outreach at once.** When a request was cancelled at minute 5, Broadcast had already alerted about 40 people, while Adaptive and Fixed had alerted about 9 to 10; no strategy created any further outreach after the cancel.

- **Adaptive is a trade-off, not a winner on every axis.** Its benefit is a much lower alert burden and visible reasoning, at the cost of time. It was not tuned to win the comparison, and these results do not show any improvement in real donation outcomes.

3.7 Limitations

- No SMS, e-mail, push or phone gateway; “alert sent” is an in-app notification record, and delivery is only evidenced when the donor opens it.
- No real hospital, blood-bank stock, GPS or government integration.
- No built-in scheduler; time-based decisions run when an administrator has the coordination pages open, through a management command, or when a donor responds.
- Simulation results come from scripted fictional donors, not measured behaviour.
- Locality coordinates are approximate and donor map positions are rounded to about 1 km.
- Threaded concurrency was verified on SQLite only.
- The decision-support assistant is rule-based, not a language model; questions outside its supported list are declined.
- There are no automated frontend tests.
- Approving a request whose required time has passed is now refused, and approval alerts only the first batch instead of every compatible donor.

3.8 Expected Outcomes

- Coordinators always know how many volunteers are still needed for each emergency.
- Donors receive fewer, more relevant alerts when others respond quickly.
- Declined and unanswered alerts are no longer confused.
- Every alert and decision can be explained and audited.
- New outreach strategies can be tested safely before being used.

CHAPTER 4

CONCLUSIONS

4.1 Conclusion

Through interviews with Dr. Chanthru, Dr. Bruno and Dr. Selvam, we found that hospitals first check the blood bank and then search for donors through social media and personal contacts, that delays happen when the required blood group is unavailable, that rare groups such as AB-negative are hard to keep in stock, and that there is no centralised donor system. Once a request goes out, nobody follows up on who has actually agreed.

From forty brainstormed ideas we selected three concepts (a smart emergency donor matching system, real-time blood bank stock and shortage monitoring, and a verified emergency blood alert network) and combined them into BloodConnect, which we extended into BloodConnect 2.0. Its Adaptive Emergency Escalation Engine ranks donors transparently, alerts small batches, recalculates the remaining need after every response, widens the search only when the recorded state calls for it, and writes the reason for every decision to a timeline. The prototype passes all 116 automated tests. In simulation with fictional donors it reduced the number of donors contacted by about 75 % in the high-response scenario, at the cost of reaching the volunteer target more slowly than a broadcast.

BloodConnect 2.0 is a coordination aid. Clinical matching, donor eligibility and blood-unit availability remain the responsibility of the hospital and the blood bank.

4.2 Future Enhancements

- A real SMS or WhatsApp gateway with delivery receipts.
- Integration with hospital and blood-bank stock systems.
- A background scheduler (for example Celery beat) for time-based decisions.
- PostgreSQL deployment with concurrency testing.
- Learning response probabilities from real data after consent and ethics review.
- An automated frontend test suite.

- An optional language-model assistant with strict grounding in recorded data.
- A mobile app for donors with push notifications.

4.3 Team Contributions

The project was carried out by a team of six students, each leading one part of the Design Thinking process while contributing to every stage. The roles are listed in Table 4.1.

Table 4.1: Team Members and Roles

Name	Register No.	Role
Freddy Matthew R	9124104041	Team Leader
Boopathi P R	9124104026	Research Lead
Harisankaran J	9124104048	Data Analysis Lead
Balakumaran E	9124104020	Problem Definition Lead
Abilash Alexander E	9124104004	Documentation Lead
Joshua Bennet	9124104063	Design Lead

4.4 Learning Outcomes

Working through the five stages of Design Thinking on a real problem gave the team the following learning outcomes:

- Talking to doctors before designing anything showed that the real difficulty is not only finding donors, but following up on who actually responds.
- Separating divergent and convergent thinking helped the team generate many ideas first and then choose the three most feasible ones with clear reasons.
- Building a working prototype exposed questions that the chart paper did not, such as how to treat an alert that nobody answers.
- Testing with simulations and automated tests taught the team to report results honestly, including where the proposed approach is slower than a simple broadcast.
- Clear wording matters in healthcare: a donor who agrees is a volunteer, not a confirmed donation, and the final decision always rests with the blood bank.

REFERENCES

[1] Brown, T. (2008). Design Thinking. Harvard Business Review, 86(6), 84–92.

[2] Hasso Plattner Institute of Design at Stanford (d.school). An Introduction to Design Thinking: Process Guide.

[3] Django Software Foundation. Django documentation. https://docs.djangoproject.com

[4] Encode OSS. Django REST Framework documentation. https://www.django-rest-framework.org

[5] Meta Open Source. React documentation. https://react.dev

[6] Vite documentation. https://vite.dev

[7] Tailwind Labs. Tailwind CSS documentation. https://tailwindcss.com/docs

[8] Leaflet and React-Leaflet documentation. https:*leafletjs.com, https:*react-leaflet.js.org

[9] SQLite documentation. https://www.sqlite.org/docs.html

[10] BloodConnect project repository: source code, README.md and docs/BLOODCONNECT_2.md (Team BloodConnect, 2026).